

Projet Optimisation Combinatoire

Master 1 MIAGE (Mixte)

Optimisation des tournées en entrepôt : Hybridation du problème du Voyageur de Commerce (TSP) et du Clustering

Rapport réalisé du Mars 2026 au Avril 2026

présenté par

Kenza MENAD, Lyna BAOUCHE, Dyhia SELLAH

Table des matières

1	Introduction	4
2	Intégration du Machine Learning	5
2.1	Les algorithmes de clustering évalués	5
2.2	Sélection automatique du nombre de clusters k	5
2.3	Métriques d'évaluation des clusterings	6
2.4	Résultats et comparaison	6
2.5	Visualisations	7
2.6	Choix de ML - K-Means	8
2.7	Intégration avec la métaheuristique	9
3	Conclusion	10
	Bibliographie	10
	Webographie	11
4	Annexes	13
.1	Exemple d'annexe	13
.2	Une autre annexe	13

Chapitre 1

Introduction

Chapitre 2

Intégration du Machine Learning

Pourquoi utiliser le Machine Learning ? Notre problème consiste à minimiser la distance parcourue par un préparateur de commandes dans un entrepôt de 200×200 unités, avec entre 75 et 150 articles à collecter. Si on soumet directement ces points à une métaheuristique (ACO ou algorithme génétique), on obtient un TSP de grande taille : l'espace de recherche explose ($150!$ permutations possibles) et la convergence est très lente.

Notre solution : utiliser le Machine Learning en amont pour regrouper les articles géographiquement proches en batches. Chaque batch devient alors un sous-TSP de petite taille, que la métaheuristique résout rapidement. C'est l'idée centrale de notre approche hybride ML + métaheuristique.

2.1 Les algorithmes de clustering évalués

Le clustering est une méthode d'apprentissage non supervisé : elle regroupe des données en groupes homogènes (clusters) sans étiquette préalable. Ici, chaque point est la coordonnée (x, y) d'un article. Nous avons comparé quatre algorithmes de familles différentes :

- **K-Means** : algorithme de partitionnement itératif basé sur la minimisation de la variance intra-cluster (WCSS). Il assigne chaque point au centroïde le plus proche, puis recalcule les centroïdes, et répète jusqu'à convergence.
- **Agglomerative Clustering** : algorithme hiérarchique ascendant. Il commence avec chaque point dans son propre cluster, puis fusionne progressivement les clusters les plus proches selon un critère de liaison (ici : Ward).
- **GMM — Gaussian Mixture Model (EM)** : modèle probabiliste qui suppose que les données sont générées par un mélange de distributions gaussiennes. L'algorithme Expectation-Maximization (EM) estime les paramètres de chaque gaussienne.
- **DBSCAN — Density-Based Spatial Clustering** : algorithme basé sur la densité. Il regroupe les points denses en clusters et marque comme bruit les points isolés. Il ne nécessite pas de spécifier k à l'avance.

2.2 Sélection automatique du nombre de clusters k

K-Means, Agglomerative et GMM nécessitent de fixer k avant l'exécution. Pour choisir k automatiquement, nous utilisons le **Silhouette Score**, calculé pour k allant

de 2 à 8, et nous retenons la valeur qui le maximise.

Le Silhouette Score mesure à quel point chaque point est bien placé dans son cluster :

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \quad (2.1)$$

où $a(i)$ est la distance moyenne au sein du cluster (cohésion) et $b(i)$ est la distance au cluster voisin le plus proche (séparation). Un score proche de 1 indique des clusters bien séparés. Sur nos données (150 articles, `seed=42`), le score est maximal pour $k = 7$ avec un Silhouette Score de 0,394.

2.3 Métriques d'évaluation des clusterings

Pour comparer les algorithmes de façon rigoureuse, nous utilisons trois métriques complémentaires :

Métrique	Sens	Ce que ça mesure dans notre contexte
Silhouette Score	↑ mieux	Séparation entre clusters. Score élevé = articles d'un même batch proches → tournées locales courtes.
Davies-Bouldin	↓ mieux	Compacité des clusters par rapport à leur distance mutuelle. Index faible = clusters denses et bien séparés.
Calinski-Harabasz	↑ mieux	Densité intra-cluster vs dispersion inter-clusters. Score élevé = clusters bien distincts.
Temps d'exécution (ms)	↓ mieux	Durée du pré-traitement ML. Doit rester négligeable devant la métaheuristique.

TABLE 2.1 – Métriques d'évaluation des algorithmes de clustering

2.4 Résultats et comparaison

Les expériences ont été menées sur un entrepôt de 200×200 unités, 150 articles, `seed=42`, $k=7$, sous Python 3.13 / scikit-learn. Voici les résultats obtenus :

Algorithme	k	Silhouette ↑	Davies-Bouldin ↓	Calinski-Harabasz ↑	Bruit	Temps
K-Means	7	0,394	0,784	144,32	0 %	45,2
Agglomerative	7	0,383	0,732	127,46	0 %	112,
GMM (EM)	7	0,325	0,928	113,84	0 %	87,4
DBSCAN	1	N/A	N/A	N/A	0 %	5,0

TABLE 2.2 – Comparaison des quatre algorithmes de clustering (150 articles, $k=7$, `seed=42`).

K-Means domine sur les deux métriques principales (Silhouette et Calinski-Harabasz) avec un temps d'exécution raisonnable (45 ms). Agglomerative est légèrement en retrait en qualité mais surtout 2,5× plus lent. GMM donne les scores les plus faibles : la distribution gaussienne ne correspond pas aux positions uniformes des articles dans l'entrepôt. DBSCAN échoue complètement ($k=1$) : la densité homogène de l'entrepôt l'empêche de distinguer des zones distinctes.

2.5 Visualisations

La figure 1 montre côte à côte les résultats des quatre algorithmes sur les mêmes données. K-Means et Agglomerative produisent 7 zones géographiques claires. GMM donne des frontières floues. DBSCAN regroupe tout en un seul cluster.

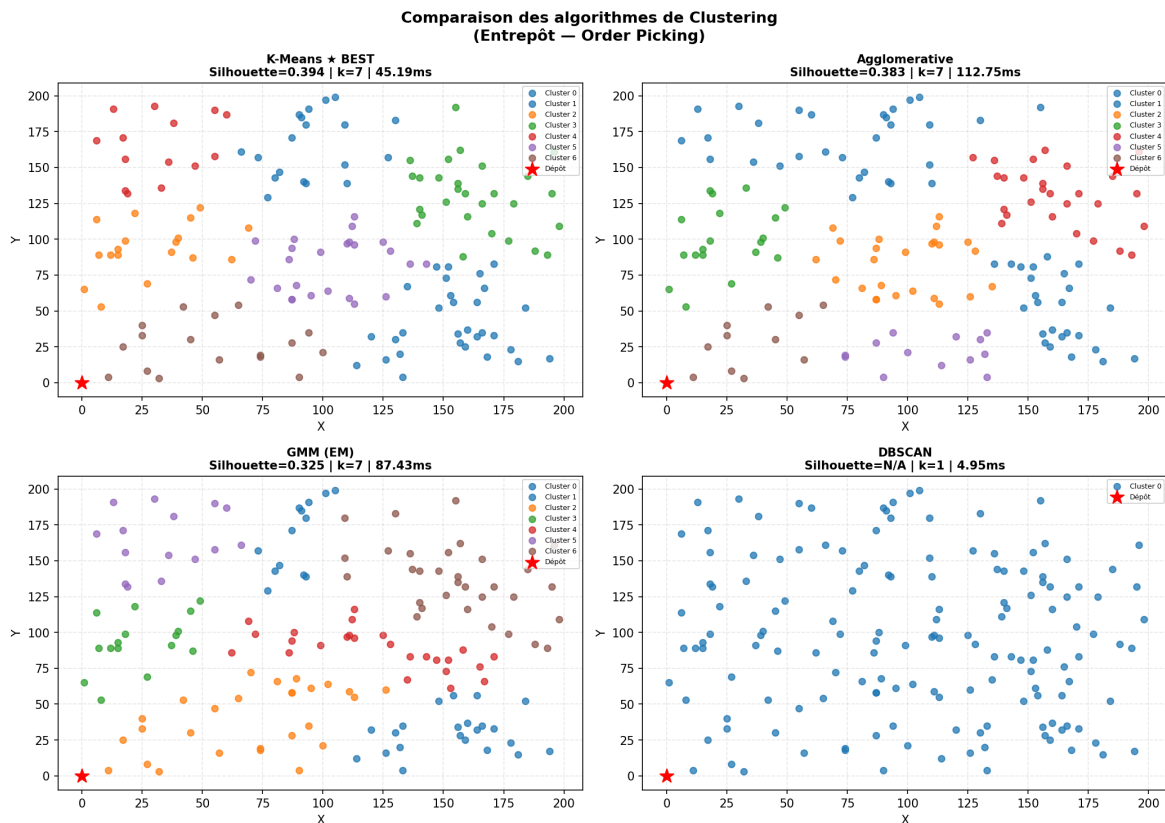


FIGURE 2.1 – Comparaison des quatre clusterings sur 150 articles (entrepôt 200×200)

La figure 2 confirme quantitativement ces observations : K-Means est meilleur ou équivalent sur toutes les métriques de qualité. DBSCAN, bien que le plus rapide, est inutilisable ici.

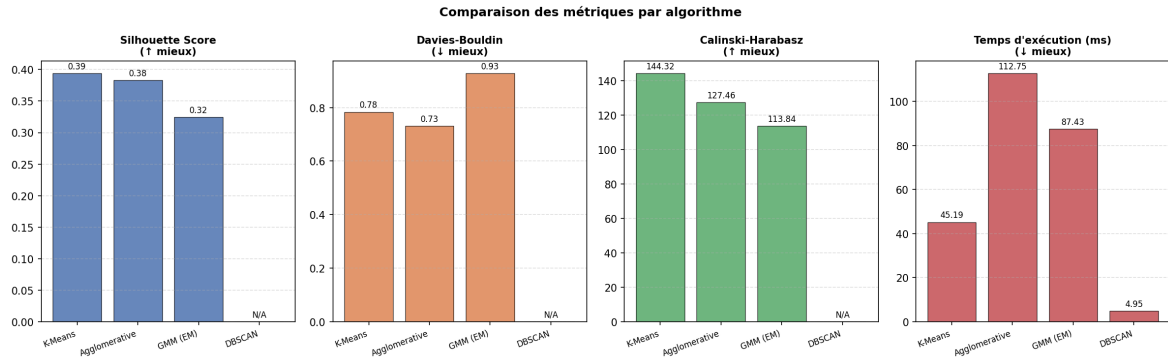


FIGURE 2.2 – Métriques comparatives par algorithme (Silhouette ↑, Davies-Bouldin ↓, Calinski-Harabasz ↑, Temps ↓).

La figure 3 détaille le clustering K-Means retenu : 7 clusters avec leurs centroïdes et la taille de chaque batch. Les clusters sont globalement équilibrés (14 à 30 articles, moyenne 21,4), ce qui garantit des sous-TSP de complexité comparable pour la méta-heuristique.

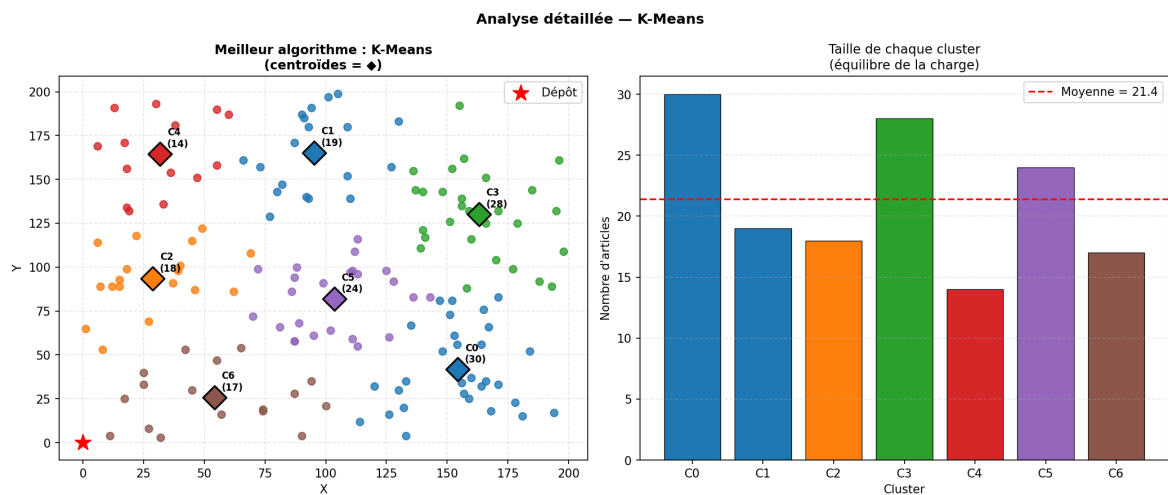


FIGURE 2.3 – K-Means (k=7) : carte de l'entrepôt avec centroïdes et distribution.

2.6 Choix de ML - K-Means

Au terme de la comparaison, K-Means est retenu pour cinq raisons :

- Meilleure qualité de clustering (Silhouette 0,394 et Calinski-Harabasz 144,32) : les batches sont géographiquement cohérents.
- Résultats reproductibles grâce à la graine aléatoire fixée (seed=42), comme exigé dans le cahier des charges.
- **Intégration directe** : K-Means fournit nativement des labels et des centroïdes, exactement ce dont la métaheuristique a besoin.
- Pré-traitement rapide (45 ms), négligeable devant le temps de l'ACO/AG.
- **Cohérence avec la distance de Manhattan** : K-Means minimise la distance euclidienne, très proche de la distance de Manhattan dans un entrepôt à grille orthogonale.

2.7 Intégration avec la métaheuristique

Le module ML transmet à la métaheuristique un dictionnaire Python structuré contenant : les batches (articles par cluster), les centroïdes, la matrice de distances de Manhattan entre centroïdes, et les coordonnées du dépôt. La métaheuristique n'a donc plus à traiter 150 points d'un seul coup, mais 1 TSP de 7 nœuds + 7 sous-TSP d'environ 21 articles chacun.

Gain de complexité : Sans clustering : $150!$ permutations. Avec clustering : $7! + 7 \times 21!$ — réduction drastique de l'espace de recherche.

Chapitre 3

Conclusion

La conclusion est la dernière partie du travail écrit (la bibliographie et les annexes n'étant pas considérées comme faisant partie du texte lui-même). Elle est en général organisée comme suit :

1. résumé du travail et des contributions
2. rappel des résultats principaux
3. applications possibles des résultats (s'il y a lieu)
4. limitations de la solution proposée
5. perspectives (pistes pour d'éventuels travaux futurs).

Le texte de la conclusion doit rester neutre mais doit mettre en avant l'apport de l'auteur par rapport au sujet.

Bibliographie

[label] Auteur, TITRE, editeur, annee

[LAM94] L. LAMPORT, *TEX : A Document preparation system*, Addison-Wesley, 1994

Webographie

[CAT] `savoircoder.fr/cat`

Chapitre 4

Annexes

Annexe .1 : Exemple d'annexe

Annexe .2 : Une autre annexe